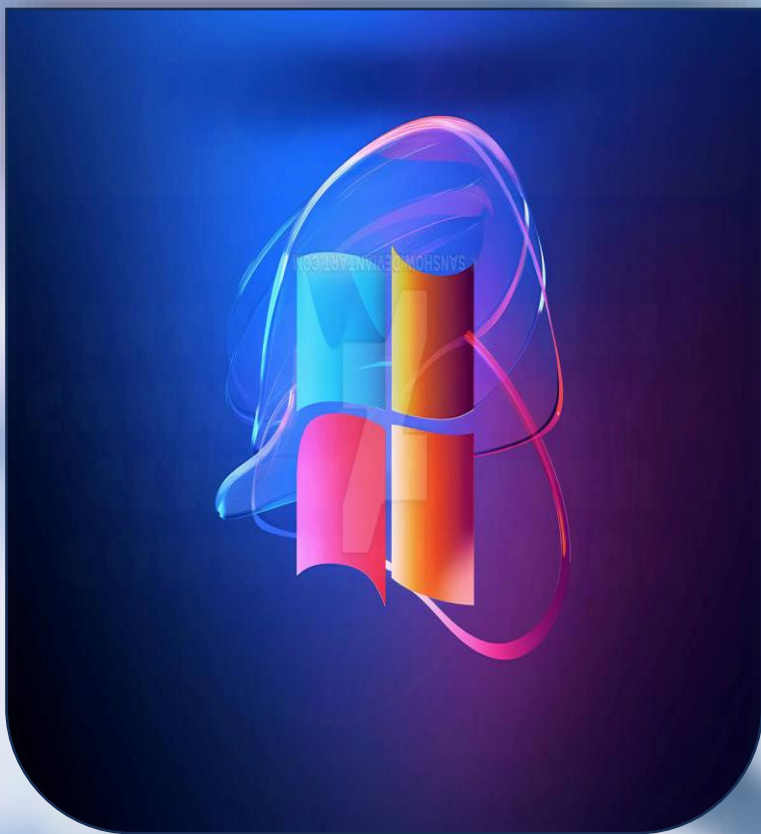
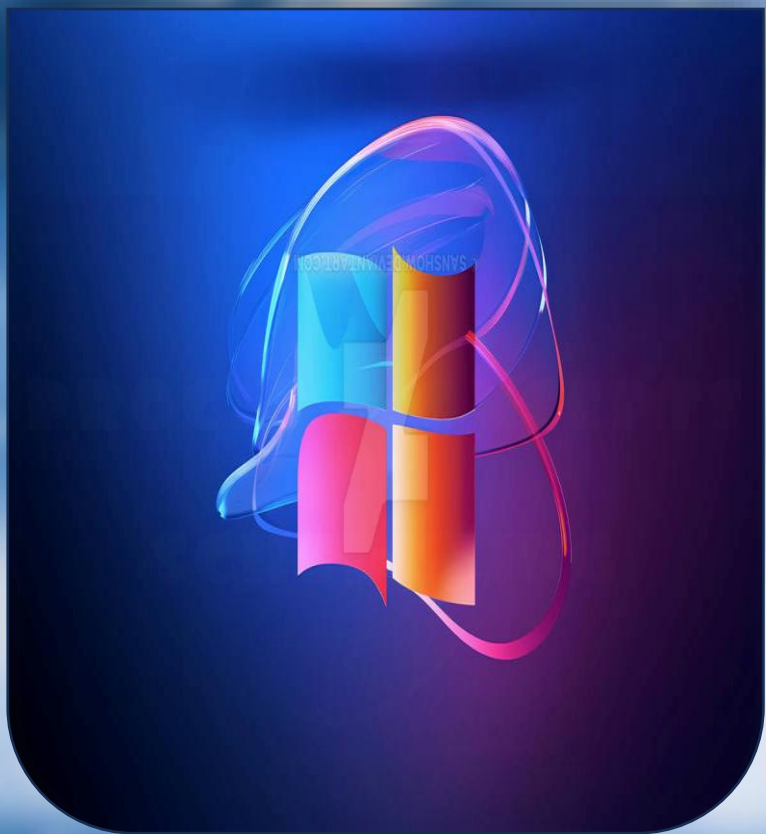




WELCOME

SEMI-FINAL TOPIC



TOPIC# 1

CPU SCHEDULING AND SCHEDULING CRITERIA



TOPIC# 1

**CPU SCHEDULING
AND
SCHEDULING CRITERIA**

TOPIC# 2

**THREAD
SCHEDULING
AND
ALGORITHMS**

TOPIC# 3



TOPIC# 1

**CPU SCHEDULING
AND
SCHEDULING CRITERIA**

TOPIC# 2

**THREAD
SCHEDULING
AND
ALGORITHMS**

TOPIC# 3

**MULTI-PROCESSOR
SCHEDULING
&
PROCESS
SYNCHRONIZATION**

TOPIC # 1 OBJECTIVES:

- DEFINE CPU SCHEDULING AND EXPLAIN ITS IMPORTANCE IN AN OPERATING SYSTEM.**
- EXPLAIN THE ROLE OF THE CPU SCHEDULER IN SELECTING PROCESSES FOR EXECUTION.**
- IDENTIFY AND DESCRIBE THE DIFFERENT SCHEDULING CRITERIA, SUCH AS:**
 - CPU UTILIZATION**
 - THROUGHPUT**
 - TURNAROUND TIME**
 - WAITING TIME**
 - RESPONSE TIME**

TOPIC # 1 OBJECTIVES:

- DIFFERENTIATE PREEMPTIVE AND NON-PREEMPTIVE SCHEDULING.**
- IDENTIFY AND EXPLAIN COMMON CPU SCHEDULING ALGORITHMS, INCLUDING:**
 - FIRST-COME, FIRST-SERVED (FCFS)**
 - SHORTEST JOB NEXT (SJN/SJF)**
 - PRIORITY SCHEDULING**
 - SHORTEST REMAINING TIME FIRST (SRTF)**
 - ROUND ROBIN (RR)**
 - MULTILEVEL QUEUE (MLQ)**

TOPIC # 1 OBJECTIVES:

- CONSTRUCT AND INTERPRET GANTT CHARTS FOR DIFFERENT SCHEDULING ALGORITHMS.**
- CALCULATE SCHEDULING PERFORMANCE MEASURES, INCLUDING WAITING TIME, TURNAROUND TIME, AND RESPONSE TIME.**
- COMPARE SCHEDULING ALGORITHMS BASED ON THEIR PERFORMANCE AND SCHEDULING CRITERIA.**
- APPLY APPROPRIATE CPU SCHEDULING ALGORITHMS TO GIVEN PROCESS SCENARIOS.**
- EVALUATE WHICH SCHEDULING ALGORITHM IS MOST SUITABLE FOR A PARTICULAR SYSTEM OR SITUATION.**

CPU SCHEDULING

**IS THE PROCESS OF
SELECTING WHICH PROCESS
FROM THE READY QUEUE
WILL BE GIVEN THE CPU
NEXT.**

**CPU SCHEDULING HELPS THE
OPERATING SYSTEM:**

- **KEEP THE CPU BUSY**
- **REDUCE WAITING TIME**
- **REDUCE TURNAROUND TIME**
- **IMPROVE RESPONSE TIME**

A. CPU UTILIZATION - PERCENTAGE OF TIME THE CPU IS BUSY.

B. THROUGHPUT - NUMBER OF PROCESSES COMPLETED
WITHIN A GIVEN PERIOD.

C. TURNAROUND TIME - THE TOTAL TIME REQUIRED FOR
A PROCESS TO FINISH, STARTING
FROM ITS ARRIVAL.

D. WAITING TIME - THE TOTAL TIME A PROCESS
SPENDS WAITING IN THE READY QUEUE.

E. RESPONSE TIME - THE AMOUNT OF TIME FROM
WHEN A PROCESS ARRIVES UNTIL IT FIRST GETS
CPU TIME.

F. FAIRNESS - THE SCHEDULER SHOULD GIVE PROCESSES
A REASONABLE OPPORTUNITY TO USE THE CPU.

**SCHEDULING
CRITERIA**

IT IS THE RULE USED BY THE CPU SCHEDULER TO DECIDE WHICH PROCESS OR THREAD WILL RUN FIRST, NEXT, AND AFTER THAT.

SCHEDULING ALGORITHM

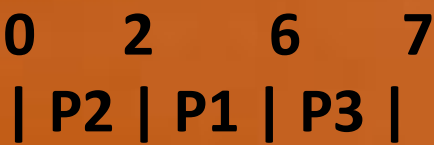
COMMON CPU SCHEDULING ALGORITHM:

- **FCFS**
- **SJN**
- **PRIORITY**
- **ROUND ROBIN**
- **MULTILEVEL QUEUE**

EXAMPLE OF FIRST COME FIRST SERVE:

PROCESS	ARRIVAL TIME (AT)	BURST TIME (BT)	COMPLETION TIME (CT)	TAT	WT
P1	1	4	6	2	1
P2	0	2	2	0	0
P3	1	1	7	6	5

Step 1: Gantt Chart- is a timeline that shows which process is running and when it starts and finishes.



Step 2: Completion Time (CT)
FORMULA:

$CT = AT + BT$

Step 3: Turnaround Time (TAT)
Formula:
 $TAT = CT - AT$

Step 4: Waiting Time (WT)

Formula:
 $WT = TAT - BT$

5. Average Waiting Time (AWT)
Formula:

$Average\ Waiting\ Time = \sum WT / \text{Number of Processes}$

6. Average Turnaround Time (ATAT)
Formula:
 $Average\ Turnaround\ Time = \sum TAT / \text{Number of Processes}$

**RULES: THE FIRST
PROCESS TO ARRIVE
THE FIRST PROCESS
TO RUN.**

EXAMPLE #2 FCFS:

PROCESS	ARRIVAL TIME (AT)	BURST TIME (BT)	COMPLETION TIME (CT)	TURNAROUND TIME (TAT)	WAITING TIME (WT)
P1	5	4			
P2	0	6			
P3	3	2			
P4	1	3			

EXAMPLE OF SJN:

PROCESS	ARRIVAL TIME (AT)	BURST TIME (BT)	COMPLETION TIME (CT)	TURNAROUND TIME (TAT)	WAITING TIME (WT)
P1	4	3	11	7	4
P2	0	6	6	6	0
P3	2	2	8	6	4
P4	1	4	15	14	10

Step 1: Gantt Chart- is a timeline that shows which process is running and when it starts and finishes.

0 6 8 11 15
| P2 | P3 | P1 | P4 |

Step 2: Completion Time (CT)

FORMULA:

$CT = AT + BT$

Step 3: Turnaround Time (TAT)

Formula:

$TAT = CT - AT$

Step 4: Waiting Time (WT)

Formula:

$WT = TAT - BT$

5. Average Waiting Time (AWT)

Formula:

$Average\ Waiting\ Time = \sum WT / \text{Number of Processes}$

6. Average Turnaround Time (ATAT)

Formula:

$Average\ Turnaround\ Time = \sum TAT / \text{Number of Processes}$

EXAMPLE # 2 OF SJN:

PROCESS	ARRIVAL TIME (AT)	BURST TIME (BT)	COMPLETION TIME (CT)	TURNAROUND TIME (TAT)	WAITING TIME (WT)
P1	0	6			
P2	0	2			
P3	0	8			
P4	0	3			

EXAMPLE OF PRIORITY SCHEDULING:

PROCESS	ARRIVAL TIME (AT)	BURST TIME (BT)	PRIORITY	COMPLETION TIME (CT)	TAT	WAITING TIME (WT)
P1	0	5	3	12	12	7
P2	0	3	1	3	3	0
P3	0	4	2	7	7	3

Step 1: Gantt Chart- is a timeline that shows which process is running and when it starts and finishes.

0 3 7 12
| P2 | P3 | P1 |

Step 2: Completion Time (CT)

FORMULA:

$$CT = AT + BT$$

Step 3: Turnaround Time (TAT)

Formula:

$$TAT = CT - AT$$

Step 4: Waiting Time (WT)

Formula:

$$WT = TAT - BT$$

5. Average Waiting Time (AWT)

Formula:

$$\text{Average Waiting Time} = \sum WT / \text{Number of Processes}$$

6. Average Turnaround Time (ATAT)

Formula:

$$\text{Average Turnaround Time} = \sum TAT / \text{Number of Processes}$$

TOPIC# 1

CPU SCHEDULING AND SCHEDULING CRITERIA



TOPIC# 1

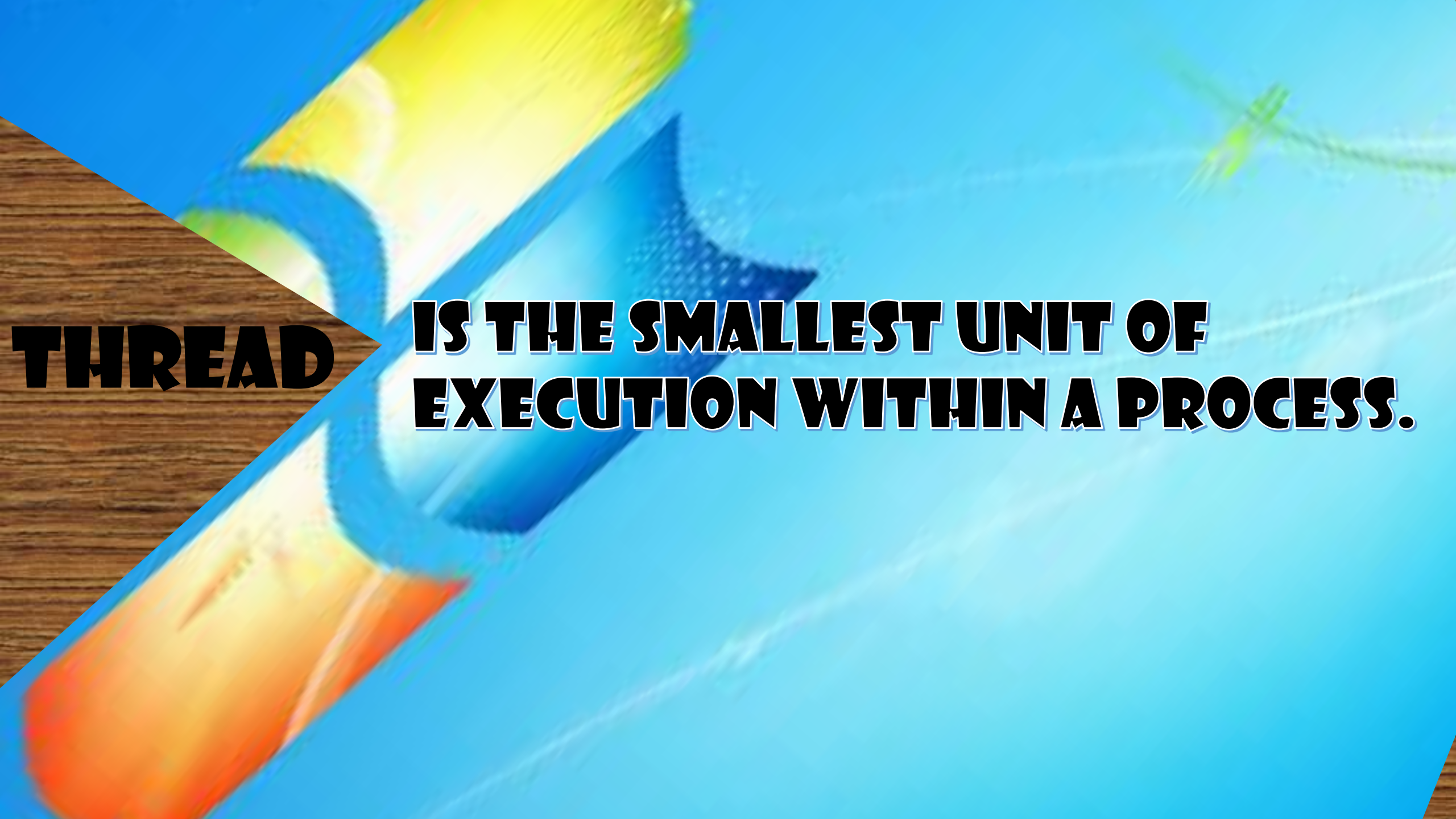
**CPU SCHEDULING
AND
SCHEDULING CRITERIA**

TOPIC# 2

**THREAD
SCHEDULING
AND
ALGORITHMS**

TOPIC# 3





THREAD

**IS THE SMALLEST UNIT OF
EXECUTION WITHIN A PROCESS.**

**IS THE PROCESS OF DECIDING WHICH
THREAD SHOULD BE GIVEN CPU TIME
AND WHEN IT SHOULD RUN.**

IT HELPS THE OPERATING SYSTEM:

- **MANAGE MULTIPLE THREADS**
- **IMPROVE CPU UTILIZATION**
- **IMPROVE APPLICATION RESPONSIVENESS**
- **ALLOW CONCURRENT EXECUTION**
- **REDUCE UNNECESSARY WAITING**
- **PROVIDE FAIRNESS AMONG THREADS**

**THREAD
SCHEDULING**

TYPES OF THREAD SCHEDULING

User- Level Thread Scheduling

Kernel-Level Thread Scheduling

Cold

TYPES OF THREAD SCHEDULING

User-Level Thread Scheduling

The threads are managed by a thread library/ user-level thread system rather than directly by the operating system kernel.

Kernel-Level Thread Scheduling

Warm

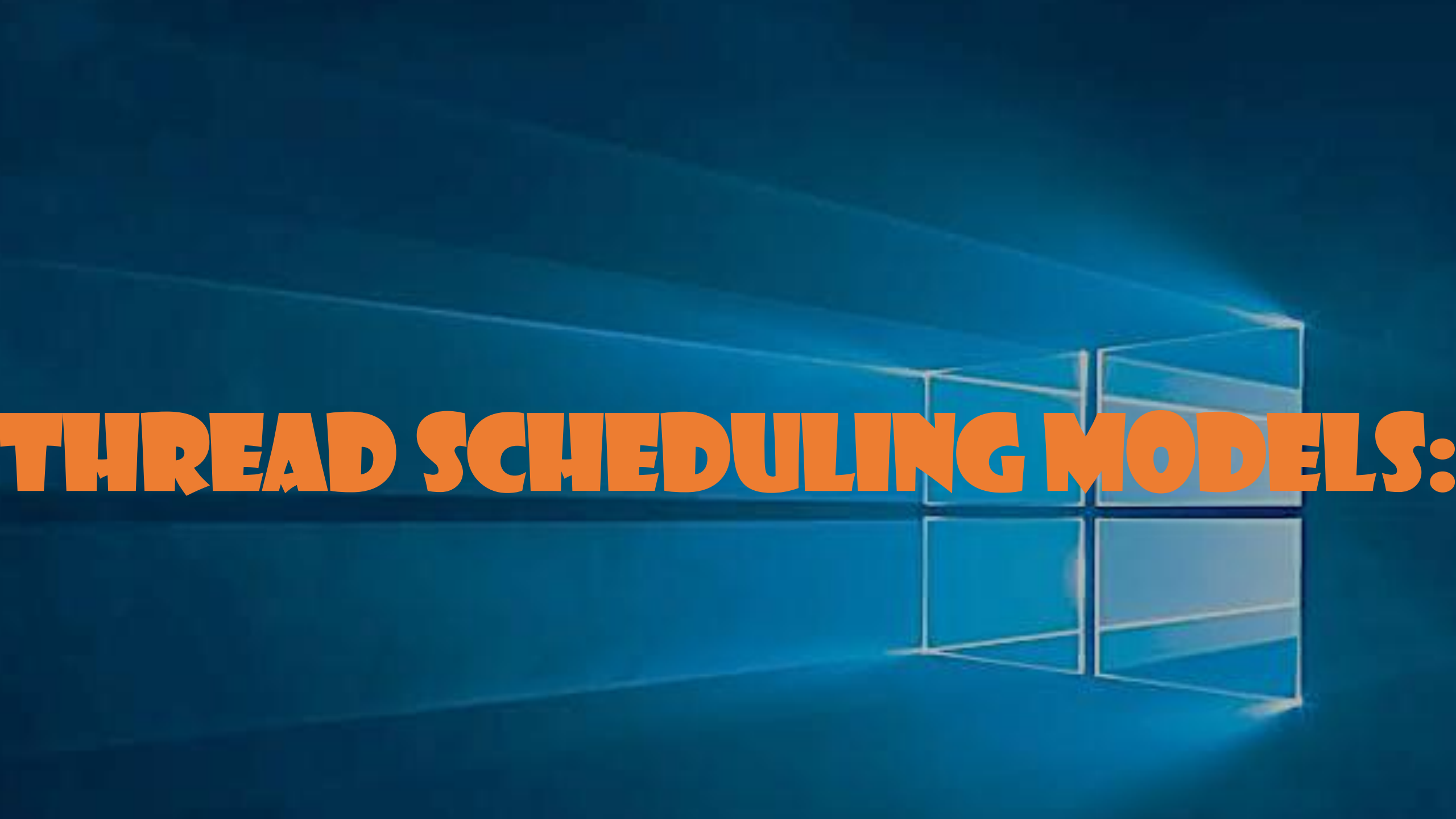
TYPES OF THREAD SCHEDULING

User-Level Thread Scheduling

The threads are managed by a thread library/ user-level thread system rather than directly by the operating system kernel.

Kernel-Level Thread Scheduling

The operating system kernel directly manages and schedules the threads.



THREAD SCHEDULING MODELS:

Many user threads
→ one kernel thread



Many-to-One



One-to-One

Each user thread

→ one kernel thread

Many user threads

→ multiple kernel threads



Many-to-Many



THREAD SCHEDULING ALGORITHM



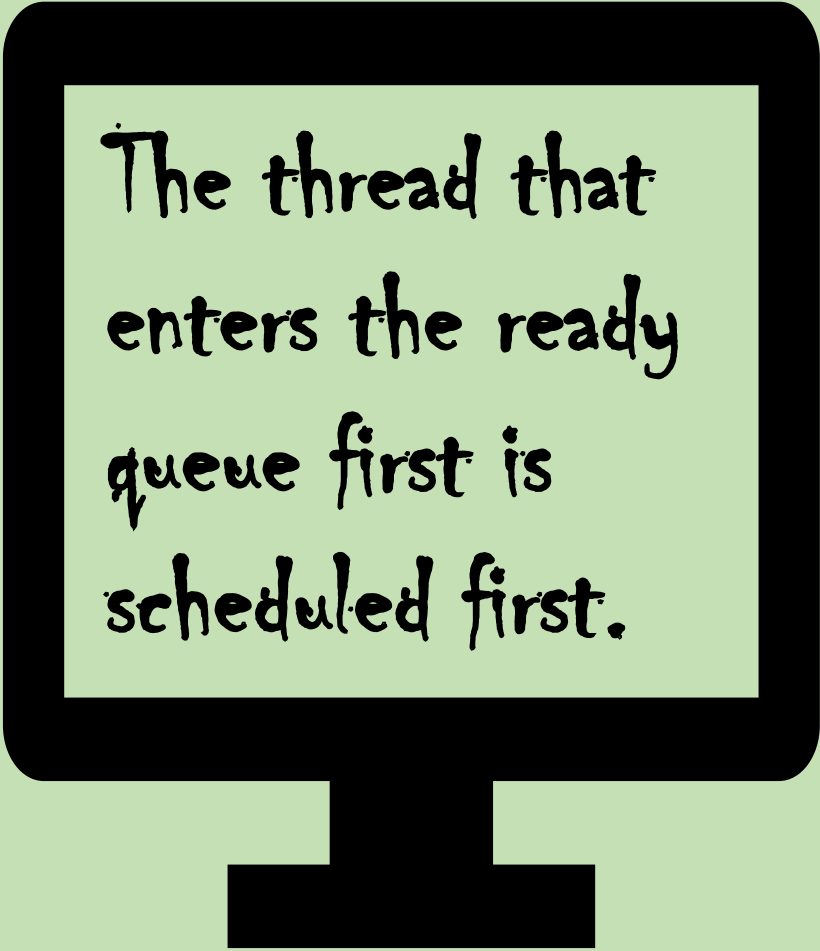
**ARE THE METHODS USED BY THE
OPERATING SYSTEM OR THREAD MANAGER
TO DECIDE WHICH THREAD SHOULD USE
THE CPU NEXT.**

COMMON THREAD SCHEDULING ALGORITHM

► FCFS

EXAMPLE:

T1 → T2 → T3 → T4



The thread that enters the ready queue first is scheduled first.

COMMON THREAD SCHEDULING ALGORITHM

► Priority Scheduling

Example:

Thread	Priority
T1	3
T2	1
T3	2

If 1 = highest priority:

T2 → T3 → T1



Threads are
assigned different
priorities.

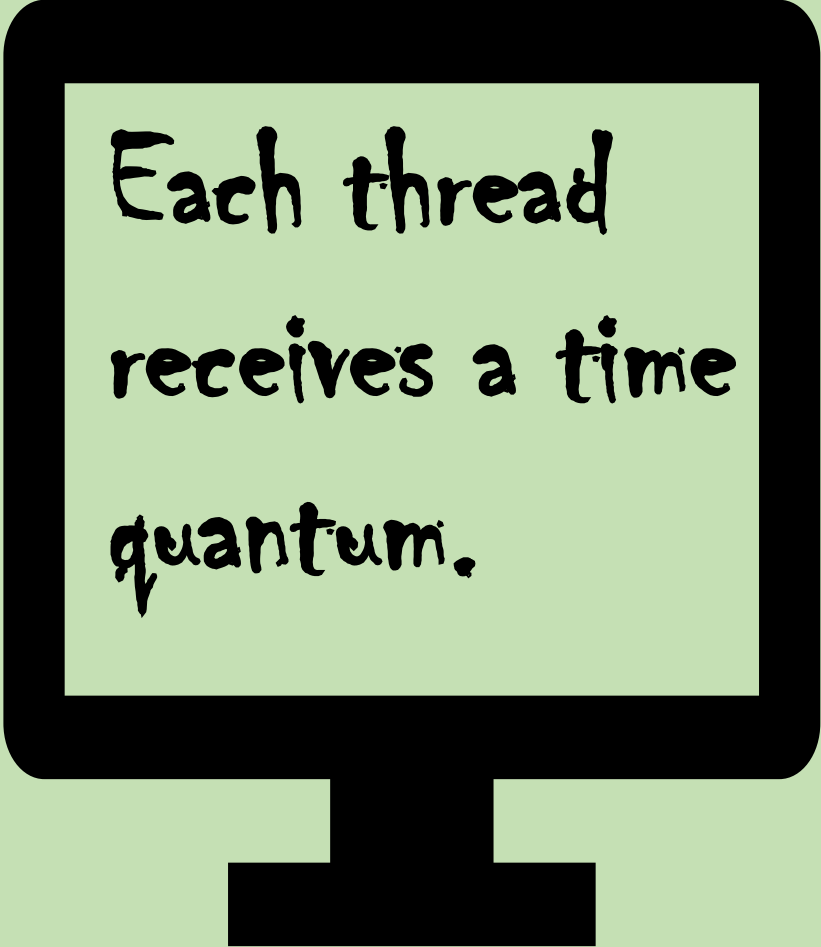
COMMON THREAD SCHEDULING ALGORITHM

► Round Robin

Example:

Time Quantum = 2

T1 → T2 → T3 → T1 → T2 → T3



Each thread
receives a time
quantum.

COMMON THREAD SCHEDULING ALGORITHM

Example:

High Priority

→ user-interface responsiveness

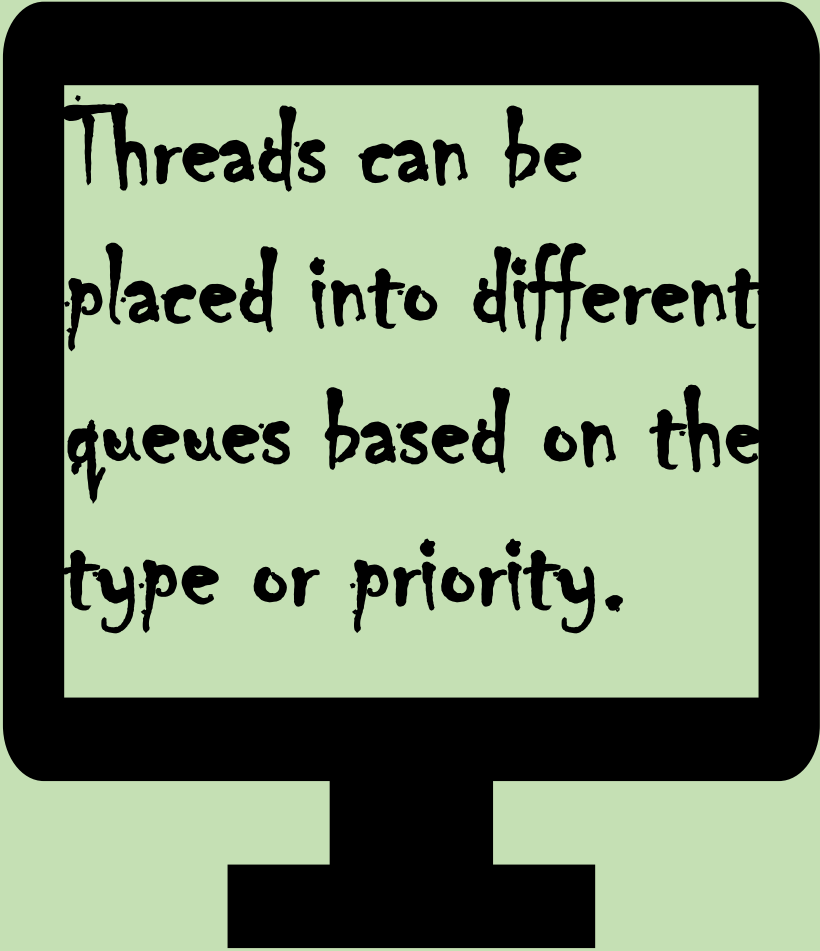
Normal Priority

→ regular application tasks

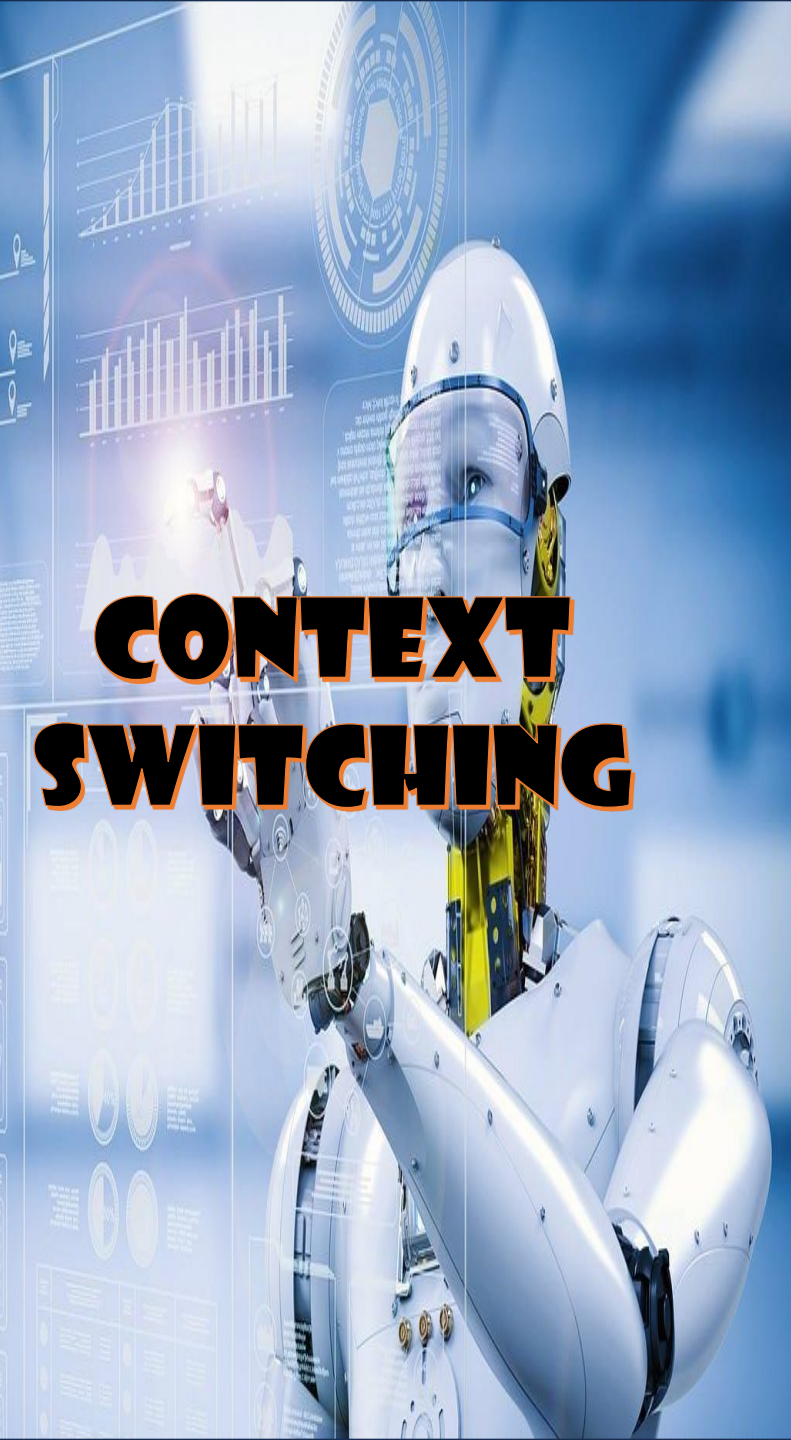
Low Priority

→ background tasks

► Multilevel Queue



Threads can be placed into different queues based on the type or priority.



CONTEXT SWITCHING

**OCCURS WHEN THE CPU STOPS EXECUTING
ONE THREAD AND SWITCHES TO ANOTHER.**



TOPIC# 1

**CPU SCHEDULING
AND
SCHEDULING CRITERIA**

TOPIC# 2

**THREAD
SCHEDULING
AND
ALGORITHMS**

TOPIC# 3



TOPIC# 1

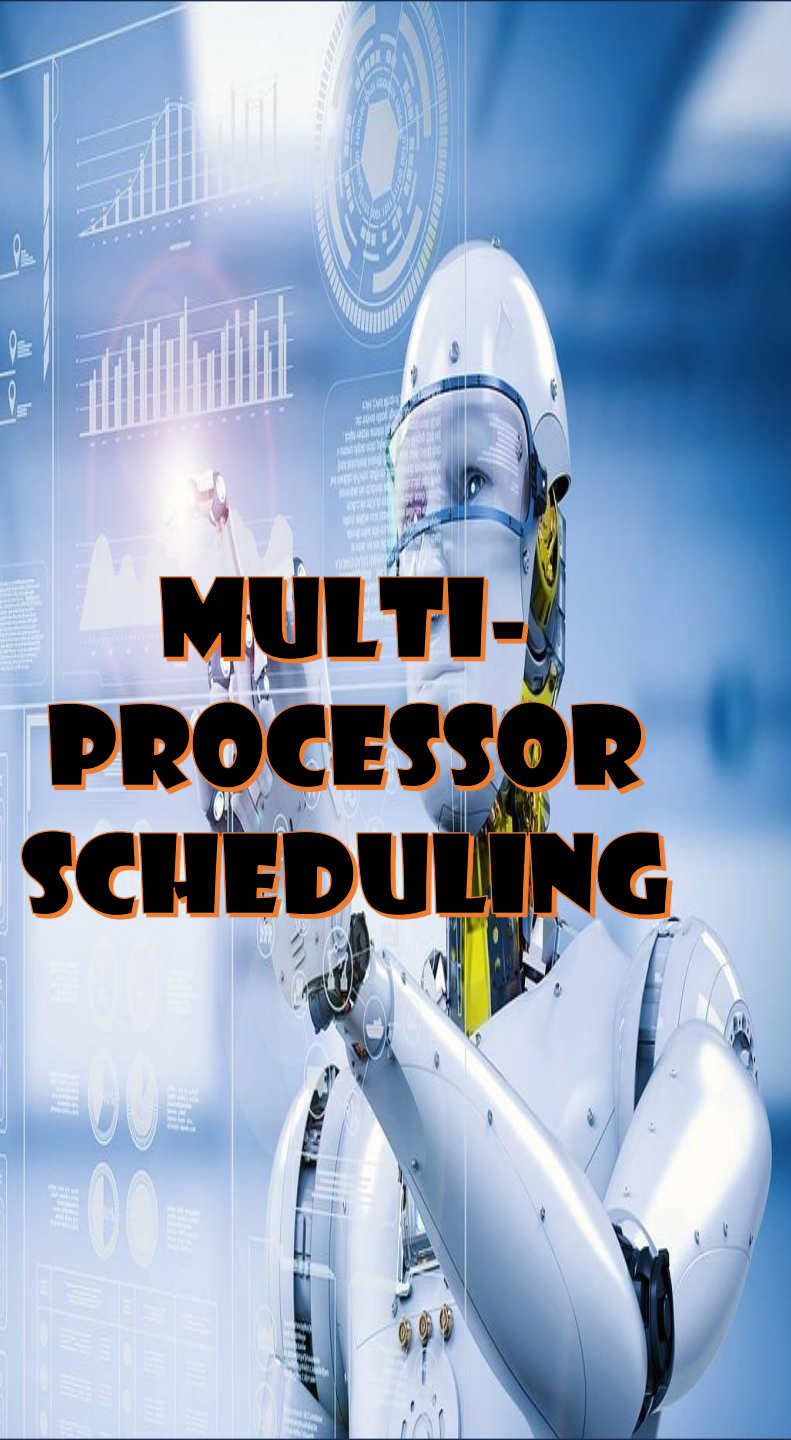
**CPU SCHEDULING
AND
SCHEDULING CRITERIA**

TOPIC# 2

**THREAD
SCHEDULING
AND
ALGORITHMS**

TOPIC# 3

**MULTI-PROCESSOR
SCHEDULING
&
PROCESS
SYNCHRONIZATION**



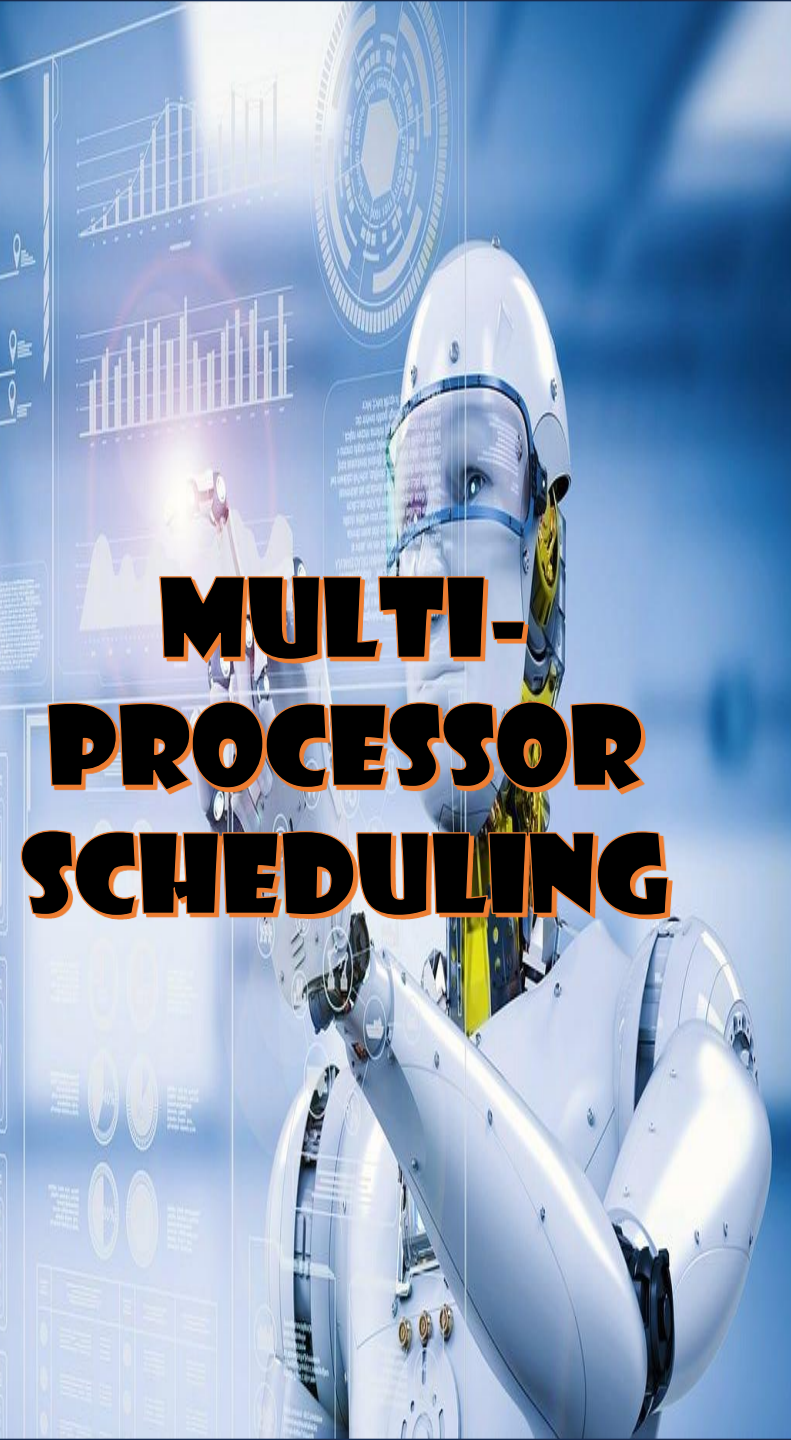
MULTI-PROCESSOR SCHEDULING

IT IS THE PROCESS OF ASSIGNING PROCESSES OR THREADS TO MULTIPLE CPUS/CORES SO THAT THEY CAN EXECUTE EFFICIENTLY.

SINGLE PROCESSOR: ONE CPU/CORE HANDLES THE PROCESSES.

MULTI-PROCESSOR: MULTIPLE CPUS/CORES CAN EXECUTE PROCESSES OR THREADS AT THE SAME TIME.





MULTI-PROCESSOR SCHEDULING

LOAD BALANCING : DISTRIBUTE WORK AMONG CPUS/CORES

CPU UTILIZATION : KEEP PROCESSORS BUSY

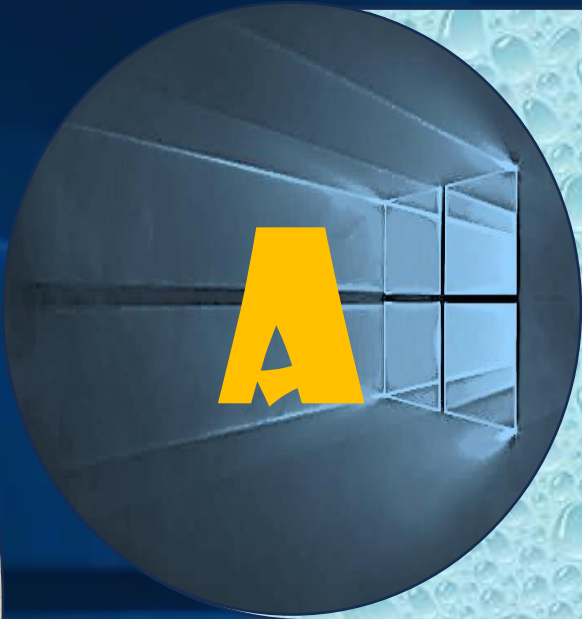
THROUGHPUT: COMPLETE MORE PROCESSES

PERFORMANCE : REDUCE EXECUTION TIME WHERE POSSIBLE

RESPONSIVENESS: IMPROVE SYSTEM RESPONSE

PROCESSOR AFFINITY: KEEP A PROCESS/ THREAD ASSOCIATED WITH A PARTICULAR CPU WHEN BENEFICIAL

TYPES OF MULTI- PROCESSOR SYSTEMS



A

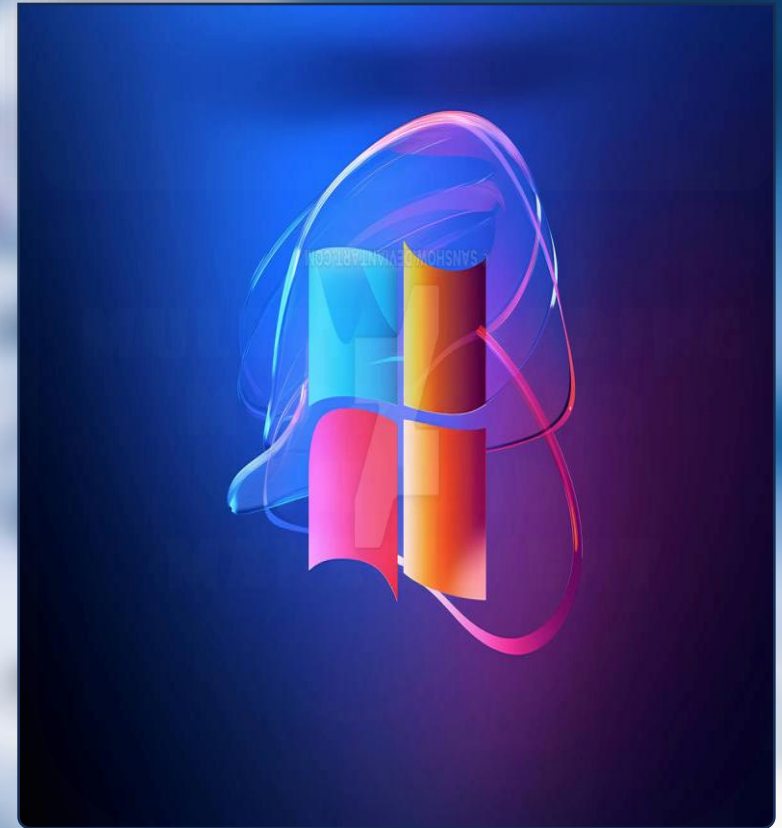
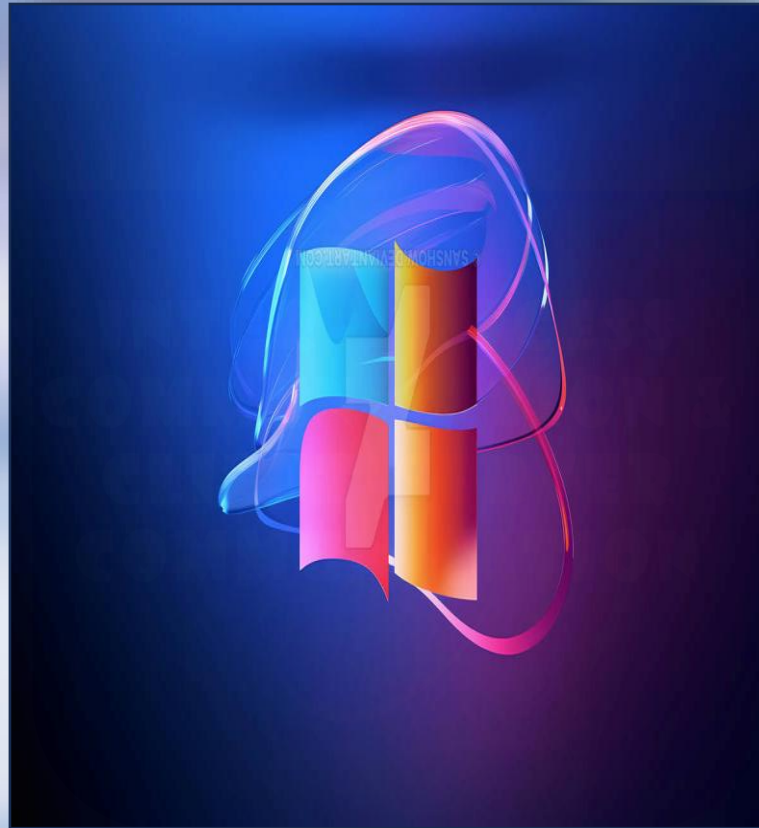
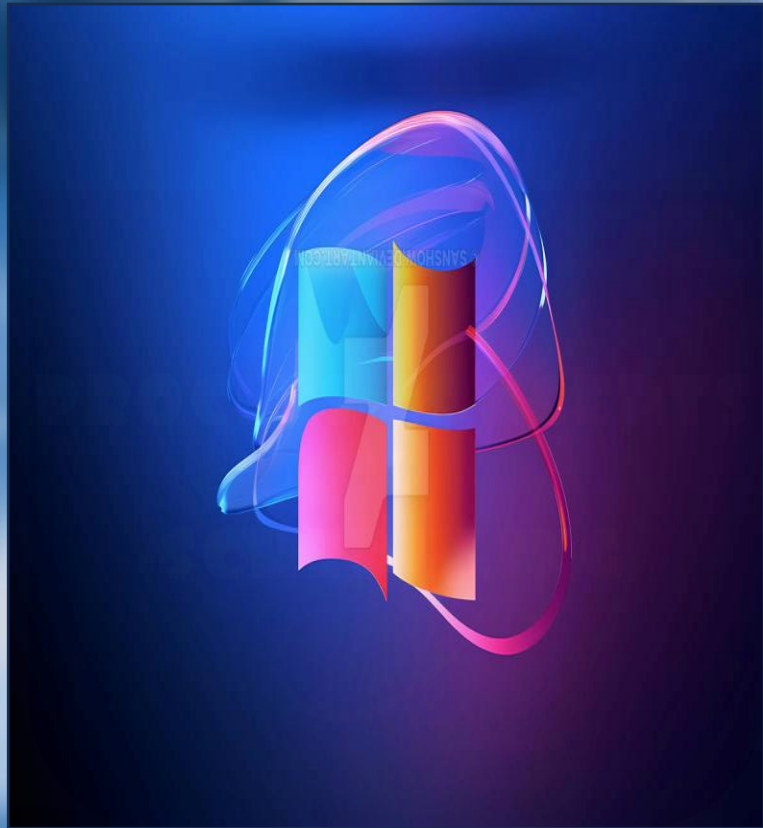
**SYMMETRIC
MULTIPROCESSING
(SMP)**



B

**ASYMMETRIC
MULTIPROCESSING
(AMP)**

MULTI-PROCESSOR SCHEDULING APPROACHES



MULTI-PROCESSOR SCHEDULING APPROACHES

LOAD SHARING

**WORK IS DISTRIBUTED
AMONG PROCESSORS SO
THAT ONE PROCESSOR
DOES NOT BECOME
OVERLOADED WHILE
ANOTHER IS IDLE.**



MULTI-PROCESSOR SCHEDULING APPROACHES

LOAD SHARING

WORK IS DISTRIBUTED AMONG PROCESSORS SO THAT ONE PROCESSOR DOES NOT BECOME OVERLOADED WHILE ANOTHER IS IDLE.

PROCESSOR AFFINITY

A PROCESS/THREAD MAY BE KEPT RUNNING ON THE SAME CPU WHEN POSSIBLE.

SOFT AFFINITY - OS TRIES TO KEEP A TASK ON THE SAME CPU BUT MAY MOVE IT.

HARD AFFINITY - TASK IS RESTRICTED TO A SPECIFIED CPU OR SET OF CPUS.



MULTI-PROCESSOR SCHEDULING APPROACHES

LOAD SHARING

WORK IS DISTRIBUTED AMONG PROCESSORS SO THAT ONE PROCESSOR DOES NOT BECOME OVERLOADED WHILE ANOTHER IS IDLE.

PROCESSOR AFFINITY

A PROCESS/THREAD MAY BE KEPT RUNNING ON THE SAME CPU WHEN POSSIBLE.

SOFT AFFINITY - OS TRIES TO KEEP A TASK ON THE SAME CPU BUT MAY MOVE IT.

HARD AFFINITY - TASK IS RESTRICTED TO A SPECIFIED CPU OR SET OF CPUS.

LOAD BALANCING

THE OPERATING SYSTEM ATTEMPTS TO DISTRIBUTE PROCESSES/THREADS EVENLY ACROSS PROCESSORS.



PROCESS SYNCHRONIZATION

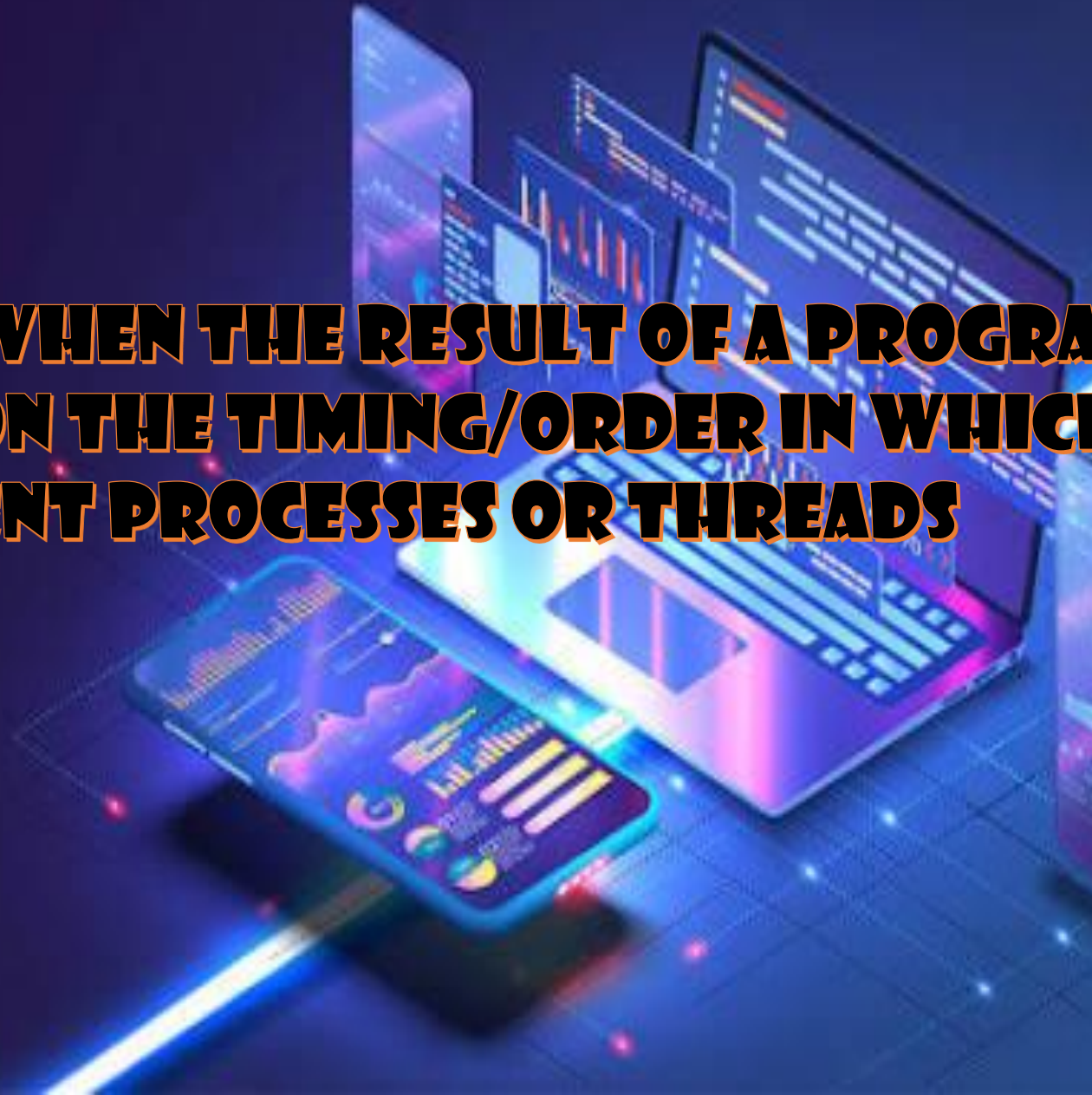
IS A MECHANISM USED TO COORDINATE PROCESSES OR THREADS SO THEY CAN SAFELY ACCESS SHARED RESOURCES AND EXECUTE IN THE CORRECT ORDER.





RACE CONDITION

**HAPPENS WHEN THE RESULT OF A PROGRAM
DEPENDS ON THE TIMING/ORDER IN WHICH
CONCURRENT PROCESSES OR THREADS
EXECUTE.**



IT IS THE PART OF A PROGRAM WHERE A PROCESS/THREAD ACCESSES SHARED DATA OR RESOURCES.

CRITICAL SECTION

REQUIREMENTS FOR A GOOD CRITICAL SECTION SOLUTION:

- 1. MUTUAL EXCLUSION** - ONLY ONE PROCESS/ THREAD CAN EXECUTE THE CRITICAL SECTION AT A TIME.
- 2. PROGRESS** - IF NO PROCESS IS INSIDE THE CRITICAL SECTION AND SOME PROCESSES WANT TO ENTER, THE SELECTION OF WHO ENTERS NEXT SHOULD NOT BE POSTPONE INDEFINITELY.
- 3. BOUNDED WAITING** - A PROCESS SHOULD NOT WAIT FOREVER TO ENTER THE CRITICAL SECTION.

SYNCHRONIZATION TOOLS

MUTEX

**ALLOWS ONLY ONE
THREAD/PROCESS TO
ACCESS A PROTECTED
RESOURCE AT A TIME.**

SEMAPHORE

**IS A
SYNCHRONIZATION
MECHANISM THAT
USES A COUNTER TO
CONTROL ACCESS TO
SHARED RESOURCES.
TWO COMMON
OPERATIONS:**

- **WAIT() / P()**
- **SIGNAL() / V()**

MONITOR

**IS A HIGHER-LEVEL
SYNCHRONIZATION
MECHANISM THAT
COMBINES SHARED
DATA AND THE
PROCEDURES THAT
OPERATE ON IT, WHILE
CONTROLLING ACCESS.**

CLASSICAL SYNCHRONIZATION PROBLEMS

PRODUCER-CONSUMER PROBLEM

**PRODUCER - CREATES DATA.
CONSUMER - USES DATA.**

READERS-WRITERS PROBLEM

**MULTIPLE READERS
MAY ACCESS SHARED
DATA,**

**WRITERS REQUIRE
EXCLUSIVE ACCESS.**

DINING PHILOSOPHERS PROBLEM

**USED TO DEMONSTRATE
CHALLENGES INVOLVING
RESOURCE ALLOCATION,
DEADLOCK, AND
SYNCHRONIZATION.**

DEADLOCK

**OCCURS WHEN PROCESSES/THREADS
ARE WAITING FOR RESOURCES HELD
BY EACH OTHER AND NONE CAN
PROCEED.**

STARVATION

**OCCURS WHEN A PROCESS/THREAD
WAITS FOR A RESOURCE OR CPU
OPPORTUNITY FOR AN EXCESSIVELY
LONG TIME BECAUSE OTHER PROCESSES
CONTINUALLY GET PRIORITY.**